Docker + Docker Compose + Volume + Network Deep Hinglish Guide

Goal: Docker Compose kya hai, container lifecycle, data persistence, volumes, bind mounts, anonymous/named volumes, networking drivers, container-to-container communication, Dockerfile, compose commands aur real MongoDB/Node examples ko deeply samajhna.

Important idea: Container delete hone par container ke andar ka writable data delete ho sakta hai. Lekin Dockerfile, docker-compose.yml aur image files alag cheezein hain. Persistent data ke liye volume ya bind mount use karte hain.

1. Docker Compose kya hota hai?

Docker Compose ek tool hai jisse hum ek application ke multiple containers ko ek YAML file me define karke ek saath create, configure, network aur run kar sakte hain.

Example application: Node.js backend + MongoDB + Mongo Express. Manually hume alag-alag docker run commands deni pad sakti hain. Compose me ye configuration mongo.yml/docker-compose.yml me likh dete hain.

```
services:
nodeapp:
build: .
ports:
- "3000:3000"
environment:
MONGO_URI: mongodb://mongo:27017/appdb
depends_on:
- mongo

mongo:
image: mongo:latest
volumes:
- mongo_data:/data/db

volumes:
mongo_data:
```

Compose kya set up karta hai? Services/containers, networks, environment variables, ports, volumes, build configuration, startup dependency aur restart behavior jaise settings ko ek declarative configuration se manage karta hai.

2. Dockerfile vs Image vs Container vs Compose YAML

Item	Simple meaning	Role
Dockerfile	Recipe / blueprint	Image banane ke instructions
Image	Ready package/template	Container create karne ka base
Container	Running/stopped instance	Application actually run hoti hai

docker-compose .yml	Multi-container configuration	Services + network + volumes + ports ko define karta hai
Volume	Persistent storage	Container delete hone ke baad bhi data rakh sakta hai

Key point: Dockerfile delete nahi hota jab container stop/delete hota hai. Dockerfile aapke project ka normal file hai. Image bhi container se logically alag hoti hai. Container delete karne se image automatically delete nahi hoti.

3. docker compose commands ko samjho

docker compose up — Compose file ke services create/start karta hai; foreground me logs dikh sakte hain.

```
docker compose up
```

docker compose up -d — Detached mode: background me run karta hai.

```
docker compose up -d
```

docker compose up --build — Start karne se pehle required images ko rebuild karta hai.

```
docker compose up --build
```

docker compose build — Compose services ke Dockerfiles se images build karta hai.

```
docker compose build
```

docker compose build --no-cache nodeapp — nodeapp image ko cache use kiye bina rebuild karta hai.

```
docker compose build --no-cache nodeapp
```

docker compose down — Compose ke containers aur default network ko remove karta hai; named volumes by default nahi remove hote.

```
docker compose down
```

docker compose down -v — Containers/network ke saath Compose-managed volumes bhi remove karta hai. Persistent DB data delete ho sakta hai.

```
docker compose down -v
```

docker compose ps — Compose services ke containers ka status.

```
docker compose ps
```

docker compose logs — Sab services ke logs.

```
docker compose logs
```

docker compose logs -f nodeapp — nodeapp ke logs live follow karta hai.

```
docker compose logs -f nodeapp
```

docker compose restart nodeapp — nodeapp container restart karta hai.

```
docker compose restart nodeapp
```

docker compose stop nodeapp — Container stop karta hai; remove nahi karta.

```
docker compose stop nodeapp
```

docker compose start nodeapp — Existing stopped container start karta hai.

```
docker compose start nodeapp
```

docker compose exec nodeapp ls -la /app/public — Running nodeapp container ke andar command execute karta hai.

```
docker compose exec nodeapp ls -la /app/public
```

docker compose exec nodeapp sh — Running container ke andar shell open karta hai.

```
docker compose exec nodeapp sh
```

docker compose run --rm nodeapp sh — Temporary container me command run; --rm ke baad temporary container remove.

```
docker compose run --rm nodeapp sh
```

docker compose pull — Compose file me declared images pull karta hai.

```
docker compose pull
```

docker compose config — Final merged/validated Compose configuration print karta hai.

```
docker compose config
```

4. Aapke commands ka exact meaning

```
docker-compose -f mongo.yml logs mongo-express
```

Old-style command syntax me **docker-compose** hyphen wala executable hai. Modern Docker Compose me generally **docker compose** use hota hai. **-f mongo.yml** batata hai ki default compose.yaml ke badle mongo.yml file use karo. **logs mongo-express** sirf mongo-express service ke logs dikhata hai.

```
docker compose -f mongo.yml up
```

mongo.yml ke andar defined services ko create/start karta hai. Agar image missing hai to pull ho sakti hai; agar build configuration hai to build bhi ho sakta hai.

```
docker build -t tarunapp:1.4 .
```

docker build Dockerfile se image banata hai. **-t tarunapp:1.4** image ka name/tag set karta hai. Final **.** build context hai—current directory Docker ko files provide karegi. Aapke command me trailing dot hona chahiye.

```
docker run -p 3000:3000 -e MONGO_URI="mongodb://tarun:tarun2805@host.docker
.internal:27017/?authSource=admin" tarunapp:1.4
```

-p 3000:3000: host port 3000 → container port 3000. **-e MONGO_URI=...**: container ke andar environment variable set. **host.docker.internal**: Docker Desktop par container se host machine ko refer karne ka convenient hostname. **tarunapp:1.4**: jis image se container banana hai.

5. Data lose kyun hota hai?

Container ka writable layer temporary nature ka hota hai. Agar MongoDB ne **/data/db** me database files container ke writable layer me rakhi hain aur aap container **remove** kar dete ho, to wo data normally container ke saath chala jata hai.

Stop ≠ Delete. Container stop karne par container ka filesystem generally bana rehta hai. Container ko remove karne par writable layer remove hoti hai. Isliye 'stop ke baad data delete' kehna exact nahi hai; delete/remove ke context ko

distinguish karna zaroori hai.

Best practice: Database ke liye named volume ya external persistent storage use karo.

```
services:
mongo:
image: mongo:latest
volumes:
- mongo_data:/data/db

volumes:
mongo_data:
```

Ab Mongo container delete/recreate karne par bhi **mongo_data** volume attached kiya ja sakta hai, isliye database files survive kar sakti hain. Lekin **docker compose down -v** ya volume deletion se data permanently lose ho sakta hai.

6. Docker Volume kya hai?

Volume Docker ka managed storage area hai jo container ke filesystem se alag rakha jata hai. Container temporary ho sakta hai, lekin volume persistent data ke liye banaya jata hai.

Without volume: MongoDB data → container writable layer → container remove → data gone.
With volume: MongoDB data → volume → container remove/recreate → same volume attach → data available.

Common database paths: MongoDB image me usually **/data/db**. Node app ke liye aap apni application directory jaise **/app** choose kar sakte ho.

7. Named volume, anonymous volume, bind mount

Type	Example	Who manages storage?	Use

Named volume	mongo_data:/data/db	Docker	DB/data persistence
Anonymous volume	/data/db	Docker, unnamed ID	Temporary/implicit persistence
Bind mount	./public:/app/public	You/host filesystem	Development, live code/assets

Named volume: name aap choose karte ho. Example **mongo_data:/data/db**. Easy backup/inspection/reuse.

Anonymous volume: Docker automatically ek generated volume name/id create karta hai. Example **- /data/db**. Explicit reuse ke liye named volume zyada clear hai.

Bind mount: host ka real directory/file container ke path par mount hota hai. Example **./public:/app/public**. Host directory me change karo to container me reflect ho sakta hai.

Host dir path = aapke Windows/Linux machine ka actual path. **Container dir path** = container ke andar ka path. **Mount path** = container ke andar woh location jahan host/volume attach hota hai.

```
services:
nodeapp:
volumes:
- ./public:/app/public
- mongo_data:/data/db
```

8. Volume commands

docker volume ls — All Docker volumes list.

```
docker volume ls
```

docker volume create mongo_data — Named volume create.

```
docker volume create mongo_data
```

docker volume inspect mongo_data — Volume ka driver, mountpoint aur metadata.

```
docker volume inspect mongo_data
```

docker volume rm mongo_data — Specific unused volume remove.

```
docker volume rm mongo_data
```

docker volume prune — Unused volumes remove; carefully use.

```
docker volume prune
```

docker volume ls -q — Only volume IDs/names output.

```
docker volume ls -q
```

docker compose down -v — Compose project ke volumes remove kar sakta hai; DB data loss risk.

```
docker compose down -v
```

9. Volume example: MongoDB persistence

```
services:
mongo:
image: mongo:latest
container_name: tarunmongo
ports:
- "27017:27017"
environment:
MONGO_INITDB_ROOT_USERNAME: tarun
MONGO_INITDB_ROOT_PASSWORD: tarun2805
volumes:
- mongo_data:/data/db

volumes:
mongo_data:
```

Flow: Mongo container start → Mongo writes database files to /data/db → /data/db is backed by mongo_data → container stop/remove → volume remains → new Mongo container attaches mongo_data → old database files available.

10. Docker Network kya hai?

Docker network containers ko ek logical communication environment deta hai. Iske through containers ek dusre ko reach kar sakte hain, usually service/container DNS name ke through.

Example: Node container aur Mongo container same custom bridge network par hain. Node ka Mongo URI host machine ke localhost par nahi, balki Mongo service name par ho sakta hai:

```
MONGO_URI=mongodb://tarun:tarun2805@mongo:27017/?authSource=admin
```

Yahan **mongo** service name/DNS name hai. Container ke andar **localhost** ka matlab usually wahi current container hota hai, host machine ya doosra container nahi.

11. Network driver kya karta hai?

Network driver decide karta hai Docker networking ka behavior: containers kis network namespace/configuration me rahenge aur traffic kaise connect hoga.

Driver	Meaning	Typical use
bridge	Docker-hosted virtual network; containers can communicate on same bridge/network.	Default/custom app networks
host	Container host ka network stack share karta hai; port isolation behavior changes.	Performance/special networking
none	Networking disabled/isolated.	Maximum network isolation

Bridge: default/common choice. Custom bridge network DNS-based service discovery ko easy banata hai.
Host: host networking use karta hai; Linux par especially relevant. Docker Desktop behavior platform-specific ho sakta hai.
None/null: container ko normal network connectivity nahi milti.

12. Custom network kyun useful hai?

```
docker network create tarun_net
docker run -d --name mongo --network tarun_net mongo
docker run -d --name nodeapp --network tarun_net tarunapp:1.4
```

Ab nodeapp container Mongo ko **mongo:27017** se reach kar sakta hai, assuming Mongo listening correctly. Custom network se application components ko logically group karna aur isolation improve karna easy hota hai.

13. Container → Host machine interaction

Container ko host ki service access karni ho to Docker Desktop par **host.docker.internal** commonly use hota hai.

```
MONGO_URI=mongodb://tarun:tarun2805@host.docker.internal:27017/?authSource=
admin
```

Aapke example me exactly isi wajah se host machine par chal raha MongoDB containerized Node app se accessible banaya ja raha hai. Port 27017 host side par reachable hona chahiye aur Mongo auth/settings correct honi chahiye.

14. Container → Container interaction

```
services:
nodeapp:
build: .
environment:
MONGO_URI: mongodb://tarun:tarun2805@mongo:27017/?authSource=admin
depends_on:
- mongo

mongo:
image: mongo:latest
```

Compose automatically project network banata hai. Services same network par hoti hain. Isliye Node → **mongo:27017**. Yahan host port publish karna container-to-container communication ke liye mandatory nahi hai; internal network enough ho sakta hai.

15. Port mapping vs network communication

-p 3000:3000 ka matlab external/host traffic ko container ke port 3000 tak publish karna. Do containers same Docker network me hain to unka internal traffic service/container port par ho sakta hai without publishing that port to host.

```
Browser -> Host:3000 -> nodeapp:3000
nodeapp -> mongo:27017
mongo -> mongo_data volume
```

16. Docker Compose complete practical stack

```
services:
nodeapp:
build: .
container_name: nodeapp
ports:
- "3000:3000"
environment:
MONGO_URI: mongodb://tarun:tarun2805@mongo:27017/?authSource=admin
depends_on:
- mongo
volumes:
- ./public:/app/public

mongo:
image: mongo:latest
container_name: tarunmongo
environment:
MONGO_INITDB_ROOT_USERNAME: tarun
MONGO_INITDB_ROOT_PASSWORD: tarun2805
ports:
- "27017:27017"
volumes:
- mongo_data:/data/db

mongo-express:
image: mongo-express:latest
ports:
- "8081:8081"
depends_on:
- mongo
```

```
volumes:
mongo_data:
```

Architecture: Browser → nodeapp:3000. Nodeapp → mongo:27017 over Compose network. Mongo → mongo_data. Developer files/assets → bind mount ./public:/app/public. Mongo Express → mongo service internally.

17. Docker container lifecycle

```
docker run ... -> create + start
docker stop NAME -> stop
docker start NAME -> start existing stopped container
docker restart NAME -> stop + start
docker rm NAME -> remove container
docker ps -> running containers
docker ps -a -> all containers
```

Important: docker stop se container ka writable layer immediately destroy nahi hota. docker rm se container remove hota hai. Image aur named volume separate objects hain.

18. Docker images / build commands

docker images — Local images list.

```
docker images
```

docker build -t tarunapp:1.4 . — Dockerfile se image build + tag.

```
docker build -t tarunapp:1.4 .
```

docker build --no-cache -t tarunapp:1.4 . — Cache completely avoid karke rebuild.

```
docker build --no-cache -t tarunapp:1.4 .
```

docker image inspect tarunapp:1.4 — Image metadata inspect.

```
docker image inspect tarunapp:1.4
```

docker image history tarunapp:1.4 — Image layers/history dekhna.

```
docker image history tarunapp:1.4
```

docker rmi tarunapp:1.4 — Image remove, if not in use.

```
docker rmi tarunapp:1.4
```

docker image prune — Unused dangling images cleanup.

```
docker image prune
```

19. Useful container commands

docker ps — Running containers.

```
docker ps
```

docker ps -a — Running + stopped containers.

```
docker ps -a
```

docker logs NAME — Container logs.

```
docker logs nodeapp
```

docker logs -f NAME — Live-follow logs.

```
docker logs -f nodeapp
```

docker exec -it NAME sh — Interactive shell.

```
docker exec -it nodeapp sh
```

docker exec NAME ls -la /app — Run one command inside.

```
docker exec nodeapp ls -la /app
```

docker inspect NAME — Container configuration/network/mount metadata.

```
docker inspect nodeapp
```

docker stats — Live CPU/RAM/network stats.

```
docker stats
```

docker top NAME — Processes inside container.

```
docker top nodeapp
```

docker port NAME — Published port mappings.

```
docker port nodeapp
```

docker rename OLD NEW — Container rename.

```
docker rename oldname newname
```

docker cp NAME:/app/log.txt . — Copy from container to host.

```
docker cp nodeapp:/app/log.txt .
```

docker cp ./file.txt NAME:/app/ — Copy host file into container.

```
docker cp ./file.txt nodeapp:/app/
```

20. Docker network commands

docker network ls — List networks.

```
docker network ls
```

docker network create tarun_net — Create custom bridge network by default.

```
docker network create tarun_net
```

docker network inspect tarun_net — Inspect driver, subnet and connected containers.

```
docker network inspect tarun_net
```

docker network connect tarun_net nodeapp — Attach existing container to network.

```
docker network connect tarun_net nodeapp
```

docker network disconnect tarun_net nodeapp — Detach container from network.

```
docker network disconnect tarun_net nodeapp
```

docker network rm tarun_net — Remove network if no longer in use.

```
docker network rm tarun_net
```

docker network prune — Remove unused networks.

```
docker network prune
```

21. docker compose vs docker run

docker run one-off/manual container creation ke liye excellent hai. Lekin multiple containers me command bahut long aur repeatable configuration difficult ho sakti hai.
Docker Compose configuration ko YAML me store karta hai, isliye same stack ko repeatedly reproduce karna easy hota hai.

```
docker run -d --name mongo -p 27017:27017 \
-e MONGO_INITDB_ROOT_USERNAME=tarun \
-e MONGO_INITDB_ROOT_PASSWORD=tarun2805 \
-v mongo_data:/data/db mongo:latest
```

Compose me wahi settings readable YAML form me aa jaati hain. Team member clone/pull karke **docker compose up** se same architecture start kar sakta hai.

22. Common mistakes

Node container me mongodb://localhost:27017 → localhost current Node container ko refer karta hai. Mongo doosre container me hai to service name **mongo** use karo.

Mongo data disappears after recreate → MongoDB ke /data/db par named volume mount karo.

docker compose down -v → Ye volumes delete kar sakta hai. Production DB ke saath dangerous.

docker exec -it f4.../bin/bash → Correct syntax: **docker exec -it CONTAINER_ID /bin/bash** ya **docker exec -it CONTAINER_ID sh**.

docker build -t tarunapp:1.4 → Build context dene ke liye usually end me **.** chahiye.

Container port vs host port confusion → -p HOST:CONTAINER. Example 8081:8081.

Container stopped = data immediately deleted → False in general. Stop preserves container; remove deletes container writable layer.

23. One-page mental model

```
PROJECT FOLDER
■
■■■ Dockerfile -> image recipe
■■■ docker-compose.yml -> multi-container blueprint
■■■ public/ -> host files
```

```
■
■■■ IMAGE
■ ■■■ tarunapp:1.4
■
■■■ CONTAINERS
■ ■■■ nodeapp
■ ■■■ mongo
■
■■■ NETWORK
■ ■■■ app network
■ ■■■ nodeapp <-> mongo
■ ■■■ DNS: mongo
■
■■■ STORAGE
■■■ named volume: mongo_data -> /data/db
■■■ bind mount: ./public -> /app/public
```

Agar container destroy ho gaya: container-specific writable data ja sakta hai. Image safe reh sakti hai. Dockerfile safe hai. Named volume alag reh sakta hai. Bind mount ka data host par rehta hai. Isi separation ki wajah se Docker practical hai.

24. Recommended workflow for your Node + Mongo project

• 1. Dockerfile banao → Node app ko image me package karo.

• 2. docker-compose.yml banao → nodeapp + mongo + optional mongo-express.

• 3. Mongo ke /data/db ko named volume do.

• 4. Node ke development files ke liye bind mount use karo, agar live editing chahiye.

• 5. Same Compose network me Node aur Mongo rakho.

• 6. Node ke MONGO_URI me container-to-container case me mongo:27017 use karo.

- 7. docker compose up --build se start karo.
- 8. docker compose logs -f nodeapp se debugging.
- 9. docker compose exec nodeapp sh se container inspect.
- 10. Production data ke liye docker compose down -v blindly mat chalao.

25. Quick command cheat sheet

```
# Compose
docker compose up
docker compose up -d
docker compose up --build
docker compose down
docker compose down -v
docker compose ps
docker compose logs
docker compose logs -f nodeapp
docker compose build
docker compose build --no-cache nodeapp
docker compose exec nodeapp sh
docker compose exec nodeapp ls -la /app/public
docker compose restart nodeapp
docker compose stop nodeapp
docker compose start nodeapp
docker compose config

# Container
docker ps
docker ps -a
docker stop NAME
docker start NAME
docker restart NAME
docker rm NAME
docker logs -f NAME
docker exec -it NAME sh
docker inspect NAME
docker stats

# Image
docker images
docker build -t tarunapp:1.4 .
```

```
docker build --no-cache -t tarunapp:1.4 .
docker image history tarunapp:1.4
docker rmi tarunapp:1.4
docker image prune

# Volume
docker volume ls
docker volume create mongo_data
docker volume inspect mongo_data
docker volume rm mongo_data
docker volume prune

# Network
docker network ls
docker network create tarun_net
docker network inspect tarun_net
docker network connect tarun_net nodeapp
docker network disconnect tarun_net nodeapp
docker network rm tarun_net
docker network prune
```

Final rule yaad rakho

Container = temporary application runtime

Image = application ka packaged template

Dockerfile = image banane ki recipe

Compose YAML = multiple services ka blueprint

Volume = persistent Docker-managed data

Bind mount = host directory ko container path se connect karna

Network = containers ke beech communication path

Port mapping = host/external traffic ko container port tak publish karna